

README.md

Event-Driven Multi-Company Job Posting Scraper

Goals

- Scrape job postings from multiple companies once per day.
- Measure job-title and job-level frequency by company and month.
- Use an existing Confluent Kafka broker as the event backbone.
- Preserve raw source data and normalized historical records.

The design uses Java Spring Boot, Azure Functions, Azure Blob Storage, Azure Cosmos DB for NoSQL, Twilio SendGrid, GitHub Actions, and an existing Confluent

Architecture overview

```
```mermaid
flowchart LR
 A["Daily Azure Timer"] --> B["Java Azure Function
Command Producer"]
 B --> C["Confluent Kafka Broker"]
 C --> T["job.scrape.commands"]

 T --> D["Spring Boot Scraper Consumer"]
 D --> E["Company Scraper Registry"]
 E --> F["HTTP Client + Jsoup"]
 F --> G["Raw HTML/JSON
Azure Blob Storage"]
 D --> H["scrape.completed event"]
 H --> C

 C --> U["job.scrape.completed"]
 U --> I["Normalizer Consumer"]
 I --> J["Canonical Job Posting"]
 J --> K["job.postings.normalized"]
 K --> C

 K --> L["Cosmos DB: job-postings"]
 K --> M["Cosmos DB: job-observations"]

 K --> N["Metrics Consumer"]
 N --> O["Cosmos DB: monthly-metrics"]
 N --> P["Daily Summary Event"]
 P --> C

 C --> V["notifications.daily"]
 V --> Q["SendGrid Email Consumer"]
 Q --> R["Twilio SendGrid"]

 S["GitHub Repository"] --> W["GitHub Actions"]
 W --> X["Build and Test"]
 W --> Y["Deploy Function and Spring App"]
 W --> Z["Deploy Bicep: Blob + Cosmos"]
```
```

Event-driven data flow

```
```text
Daily timer
-> publish one scrape command per company
-> scraper consumer fetches search and detail pages
-> raw responses are written to Blob Storage
-> scrape-completed event is published
-> normalizer creates canonical job documents
-> normalized events update Cosmos DB
-> metrics consumer updates monthly aggregates
-> daily summary event is published
-> email consumer sends the SendGrid notification
```
```

The Kafka broker carries commands and facts. Blob Storage is the source-of-record for raw responses, while Cosmos DB stores queryable normalized data and metrics

Components

| Component | Responsibility |
|--------------------------------|--|
| Azure Timer-triggered Function | Publishes daily scrape commands |
| Spring Boot Kafka consumers | Scraping, normalization, metrics, and notifications |
| Confluent Kafka | Durable event transport and replayable processing pipeline |
| Azure Blob Storage | Raw HTML, JSON responses, and run artifacts |
| Azure Cosmos DB for NoSQL | Current postings, observations, and monthly metrics |
| Twilio SendGrid | Daily scrape summary email |
| GitHub Actions | Build, test, and deployment automation |
| Bicep | Repeatable Azure infrastructure |

Topic design

Use a clear event naming convention and keep commands separate from facts.

| Topic | Producer | Consumer | Key | Purpose |
|-------------------------|------------------|-----------------------------------|-------------------------|--|
| job.scrape.commands | Daily scheduler | Scraper workers | companyId | Requests one company scrape |
| job.scrape.completed | Scraper workers | Normalizer | companyId | Reports raw scrape completion and Blob locations |
| job.postings.normalized | Normalizer | Persistence and metrics consumers | companyId:externalJobId | Publishes canonical postings |
| job.metrics.monthly | Metrics consumer | Reporting and summary consumers | companyId:yyyy-MM | Publishes updated monthly counts |
| notifications.daily | Summary producer | Email consumer | runId | Requests the daily email |

Create a dead-letter topic for each processing topic, for example job.scrape.commands.DLT. Include retry headers or a retry-topic policy so temporary failures

Event envelope:

```
```json
{
 "eventId": "uuid",
 "eventType": "JobPostingNormalized",
 "eventVersion": 1,
 "occurredAt": "2026-08-15T10:00:00Z",
 "producer": "job-normalizer",
 "correlationId": "scrape-run-id",
 "companyId": "mckesson",
 "payload": {}
},
...
```
```

Use stable keys to preserve ordering per company or posting. Configure consumers with explicit consumer-group names so each logical stage receives each event.

Confluent Kafka topic reference

Cluster

| Setting | Value |
|-------------------------|---|
| Cluster name | `shared-kafka-dev-us-central1` |
| Cloud provider / region | Azure / South Central US |
| Bootstrap server | `pkc-57jzz.southcentralus.azure.confluent.cloud:9092` |
| Security protocol | `SASL_SSL` |
| SASL mechanism | `PLAIN` |

Topic creation settings (all topics)

Every topic was created with the following shared baseline. Increase partitions only when throughput or parallelism requires it.

| Setting | Value |
|--------------------|--------------------------|
| Partitions | `1` |
| Cleanup policy | `Delete` |
| Retention time | `604800000 ms` (7 days) |
| Retention size | `-1` (unlimited by size) |
| Max message bytes | `1048576` (1 MB) |
| Replication factor | Cluster default |

Primary topics

| Topic | Message key | Purpose |
|---------------------------|---------------------------|--|
| `job.scrape.commands` | `companyId` | Requests one company scrape per run |
| `job.scrape.completed` | `companyId` | Reports scrape completion and Blob paths |
| `job.postings.normalized` | `companyId:externalJobId` | Publishes canonical job posting events |
| `job.metrics.monthly` | `companyId:yyyy-MM` | Publishes updated monthly metric counts |
| `notifications.daily` | `runId` | Requests the daily summary email |

Dead-letter topics (DLT)

Each primary topic has a corresponding DLT. Messages are routed to the DLT after all retry attempts are exhausted.

| DLT topic | Source topic | Message key |
|-------------------------------|---------------------------|--|
| `job.scrape.commands.DLT` | `job.scrape.commands` | `companyId` (same as source) |
| `job.scrape.completed.DLT` | `job.scrape.completed` | `companyId` (same as source) |
| `job.postings.normalized.DLT` | `job.postings.normalized` | `companyId:externalJobId` (same as source) |
| `job.metrics.monthly.DLT` | `job.metrics.monthly` | `companyId:yyyy-MM` (same as source) |
| `notifications.daily.DLT` | `notifications.daily` | `runId` (same as source) |

Standard event envelope

Every value message uses this common outer envelope. The `payload` field contains topic-specific data.

```
```json
{
 "eventId": "uuid",
 "eventType": "ScrapeCompanyCommand",
 "eventVersion": 1,
 "occurredAt": "2026-08-16T03:00:00Z",
 "producer": "job-scrape-function",
 "correlationId": "run-2026-08-16",
 "companyId": "mckesson",
 "payload": {}
},
...
```
```

Data contracts (JSON Schema – Confluent Schema Registry)

Use ****JSON Schema Draft 07**** for all contracts. Schema context: `default`. Register each ``-key`` and ``-value`` subject separately under the topic's ****Data contracts****.

```
#### `job.scrape.commands-key`

```json
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "JobScrapeCommandsKey",
 "description": "Kafka message key for job.scrape.commands; keyed by companyId to preserve per-company ordering.",
 "type": "string",
 "minLength": 1,
 "maxLength": 128,
 "pattern": "^[a-z0-9][a-z0-9-]*$",

```

```
 "examples": ["mckesson", "gm-financial", "lockheed-martin"]
 }
 ...
```

```
`job.scrape.commands-value`
```

```
```json
```

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "ScrapeCompanyCommandEnvelope",
  "description": "Event envelope for requesting one company scrape run.",
  "type": "object",
  "additionalProperties": false,
  "required": ["eventId", "eventType", "eventVersion", "occurredAt", "producer", "correlationId", "companyId", "payload"],
  "properties": {
    "eventId": { "type": "string", "minLength": 1 },
    "eventType": { "type": "string", "const": "ScrapeCompanyCommand" },
    "eventVersion": { "type": "integer", "minimum": 1 },
    "occurredAt": { "type": "string", "format": "date-time" },
    "producer": { "type": "string", "minLength": 1 },
    "correlationId": { "type": "string", "minLength": 1 },
    "companyId": { "type": "string", "minLength": 1, "maxLength": 128, "pattern": "^[a-z0-9][a-z0-9-]*$" },
    "payload": {
      "type": "object",
      "additionalProperties": false,
      "required": ["runId", "companyId", "requestedAt", "attempt"],
      "properties": {
        "runId": { "type": "string", "minLength": 1 },
        "companyId": { "type": "string", "minLength": 1, "maxLength": 128, "pattern": "^[a-z0-9][a-z0-9-]*$" },
        "requestedAt": { "type": "string", "format": "date-time" },
        "attempt": { "type": "integer", "minimum": 1 }
      }
    }
  }
}
...
---
```

```
#### `job.scrape.completed-key`
```

```
```json
```

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "JobScrapeCompletedKey",
 "description": "Kafka message key for job.scrape.completed; keyed by companyId.",
 "type": "string",
 "minLength": 1,
 "maxLength": 128
}
...

```

```
`job.scrape.completed-value`
```

```
```json
```

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "JobScrapeCompletedValue",
  "type": "object",
  "additionalProperties": false,
  "required": ["eventId", "eventType", "eventVersion", "occurredAt", "producer", "correlationId", "companyId", "payload"],
  "properties": {
    "eventId": { "type": "string", "minLength": 1 },
    "eventType": { "type": "string", "const": "ScrapeCompletedEvent" },
    "eventVersion": { "type": "integer", "minimum": 1 },
    "occurredAt": { "type": "string", "format": "date-time" },
    "producer": { "type": "string", "minLength": 1 },
    "correlationId": { "type": "string", "minLength": 1 },
    "companyId": { "type": "string", "minLength": 1 },
    "payload": {
      "type": "object",
      "additionalProperties": false,
      "required": ["runId", "companyId", "startedAt", "completedAt", "searchPagesFetched", "detailPagesFetched", "blobPaths", "errors"],
      "properties": {
        "runId": { "type": "string", "minLength": 1 },
        "companyId": { "type": "string", "minLength": 1 },
        "startedAt": { "type": "string", "format": "date-time" },
        "completedAt": { "type": "string", "format": "date-time" },
        "searchPagesFetched": { "type": "integer", "minimum": 0 },
        "detailPagesFetched": { "type": "integer", "minimum": 0 },
        "blobPaths": { "type": "array", "items": { "type": "string", "minLength": 1 } },
        "errors": { "type": "array", "items": { "type": "string" } }
      }
    }
  }
}
...
---
```

```
#### `job.postings.normalized-key`
```

```
```json
```

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "JobPostingsNormalizedKey",
 "description": "Kafka message key for job.postings.normalized; format is companyId:externalJobId.",
 "type": "string",
}
```

```

 "pattern": "^[a-z0-9-]+:[^:]+$"
}
...

`job.postings.normalized-value`

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "JobPostingsNormalizedValue",
  "type": "object",
  "additionalProperties": false,
  "required": ["eventId", "eventType", "eventVersion", "occurredAt", "producer", "correlationId", "companyId", "payload"],
  "properties": {
    "eventId": { "type": "string", "minLength": 1 },
    "eventType": { "type": "string", "const": "JobPostingNormalized" },
    "eventVersion": { "type": "integer", "minimum": 1 },
    "occurredAt": { "type": "string", "format": "date-time" },
    "producer": { "type": "string", "minLength": 1 },
    "correlationId": { "type": "string", "minLength": 1 },
    "companyId": { "type": "string", "minLength": 1 },
    "payload": {
      "type": "object",
      "additionalProperties": false,
      "required": ["id", "companyId", "externalJobId", "titleRaw", "titleNormalized", "jobLevel", "location", "datePosted", "firstSeenAt", "lastSeenAt", "isActive", ],
      "properties": {
        "id": { "type": "string", "minLength": 1 },
        "companyId": { "type": "string", "minLength": 1 },
        "externalJobId": { "type": "string", "minLength": 1 },
        "titleRaw": { "type": "string", "minLength": 1 },
        "titleNormalized": { "type": "string", "minLength": 1 },
        "jobLevel": { "type": "string", "minLength": 1 },
        "location": { "type": "string", "minLength": 1 },
        "datePosted": { "type": "string", "format": "date" },
        "firstSeenAt": { "type": "string", "format": "date-time" },
        "lastSeenAt": { "type": "string", "format": "date-time" },
        "isActive": { "type": "boolean" },
        "contentHash": { "type": "string", "minLength": 1 },
        "schemaVersion": { "type": "integer", "minimum": 1 }
      }
    }
  }
}
...

---

#### `job.metrics.monthly-key`

```json
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "JobMetricsMonthlyKey",
 "description": "Kafka message key for job.metrics.monthly; format is companyId:yyyy-MM.",
 "type": "string",
 "pattern": "^[a-z0-9-]+:[0-9]{4}-[0-9]{2}$"
}
...

`job.metrics.monthly-value`

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "JobMetricsMonthlyValue",
  "type": "object",
  "additionalProperties": false,
  "required": ["eventId", "eventType", "eventVersion", "occurredAt", "producer", "correlationId", "companyId", "payload"],
  "properties": {
    "eventId": { "type": "string", "minLength": 1 },
    "eventType": { "type": "string", "const": "MonthlyMetricUpdated" },
    "eventVersion": { "type": "integer", "minimum": 1 },
    "occurredAt": { "type": "string", "format": "date-time" },
    "producer": { "type": "string", "minLength": 1 },
    "correlationId": { "type": "string", "minLength": 1 },
    "companyId": { "type": "string", "minLength": 1 },
    "payload": {
      "type": "object",
      "additionalProperties": false,
      "required": ["companyId", "month", "normalizedTitle", "jobLevel", "postingCount", "uniquePostingCount", "lastCalculatedAt"],
      "properties": {
        "companyId": { "type": "string", "minLength": 1 },
        "month": { "type": "string", "pattern": "^[0-9]{4}-[0-9]{2}$" },
        "normalizedTitle": { "type": "string", "minLength": 1 },
        "jobLevel": { "type": "string", "minLength": 1 },
        "postingCount": { "type": "integer", "minimum": 0 },
        "uniquePostingCount": { "type": "integer", "minimum": 0 },
        "lastCalculatedAt": { "type": "string", "format": "date-time" }
      }
    }
  }
}
...

---

#### `notifications.daily-key`

```

```

```json
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "NotificationsDailyKey",
 "description": "Kafka message key for notifications.daily; keyed by runId.",
 "type": "string",
 "minLength": 1
}
...

`notifications.daily-value`

```json
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "NotificationsDailyValue",
  "type": "object",
  "additionalProperties": false,
  "required": ["eventId", "eventType", "eventVersion", "occurredAt", "producer", "correlationId", "companyId", "payload"],
  "properties": {
    "eventId": { "type": "string", "minLength": 1 },
    "eventType": { "type": "string", "const": "DailyNotificationRequested" },
    "eventVersion": { "type": "integer", "minimum": 1 },
    "occurredAt": { "type": "string", "format": "date-time" },
    "producer": { "type": "string", "minLength": 1 },
    "correlationId": { "type": "string", "minLength": 1 },
    "companyId": { "type": "string", "minLength": 1 },
    "payload": {
      "type": "object",
      "additionalProperties": false,
      "required": ["runId", "summaryDate", "companiesProcessed", "newCount", "changedCount", "inactiveCount", "failures"],
      "properties": {
        "runId": { "type": "string", "minLength": 1 },
        "summaryDate": { "type": "string", "format": "date" },
        "companiesProcessed": { "type": "integer", "minimum": 0 },
        "newCount": { "type": "integer", "minimum": 0 },
        "changedCount": { "type": "integer", "minimum": 0 },
        "inactiveCount": { "type": "integer", "minimum": 0 },
        "failures": { "type": "array", "items": { "type": "string" } }
      }
    }
  }
}
...

---

#### DLT contract pattern

All DLT topics share the same value schema structure. Only `originalTopic` differs per topic.

***.DLT-key*** – same key type as the source topic (see source topic key contract above).

***.DLT-value*** – envelope that wraps the original failed event alongside failure metadata:

```json
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "DeadLetterEnvelope",
 "type": "object",
 "additionalProperties": false,
 "required": ["failedAt", "consumerGroup", "originalTopic", "originalPartition", "originalOffset", "originalKey", "retryCount", "errorType", "errorMessage", "originalEvent"],
 "properties": {
 "failedAt": { "type": "string", "format": "date-time" },
 "consumerGroup": { "type": "string", "minLength": 1 },
 "originalTopic": {
 "type": "string",
 "enum": [
 "job.scrape.commands",
 "job.scrape.completed",
 "job.postings.normalized",
 "job.metrics.monthly",
 "notifications.daily"
]
 },
 "originalPartition": { "type": "integer", "minimum": 0 },
 "originalOffset": { "type": "integer", "minimum": 0 },
 "originalKey": { "type": "string", "minLength": 1 },
 "retryCount": { "type": "integer", "minimum": 0 },
 "errorType": { "type": "string", "minLength": 1 },
 "errorMessage": { "type": "string", "minLength": 1 },
 "originalEvent": { "type": "object" }
 }
}
...

```

For each DLT subject replace the `originalTopic` `enum` with a single `const` matching the source topic:

```

| DLT subject | `originalTopic` const value |
|---|---|
| `job.scrape.commands.DLT-value` | `job.scrape.commands` |
| `job.scrape.completed.DLT-value` | `job.scrape.completed` |
| `job.postings.normalized.DLT-value` | `job.postings.normalized` |
| `job.metrics.monthly.DLT-value` | `job.metrics.monthly` |
| `notifications.daily.DLT-value` | `notifications.daily` |

```

## Stages

### ### Stage 1 â€” Java Spring Boot Azure Function foundation

1. Create the Maven Java Azure Functions project.
2. Use Java 21 consistently in the local JDK, `pom.xml`, and Azure runtime configuration.
3. Add Spring Boot dependencies and initialize the Spring context once per function instance.
4. Keep the Azure Timer function thin; it should publish commands rather than scrape directly.
5. Add local configuration and environment-variable handling.
6. Add unit-test and fixture directories.

The Spring Boot application should expose separate services for the command producer, scraper consumer, normalizer, metrics consumer, and email consumer. /

### ### Stage 2 â€” Kafka configuration and topics

1. Confirm the existing Confluent broker address, security protocol, SASL mechanism, username, and password or API key.
2. Store broker credentials in local environment variables or secret storage; never commit them.
3. Configure `bootstrap.servers`, `security.protocol`, `sasl.mechanism`, and the SASL credentials.
4. Create the topics listed above with an appropriate partition count and replication factor for the existing broker.
5. Create separate consumer groups for scraping, normalization, persistence, metrics, and email.
6. Configure `acks=all`, idempotent producers, retries, and delivery timeouts.
7. Configure consumer `enable.auto.commit=false` and commit offsets only after successful processing.
8. Configure retry and dead-letter handling.
9. Add a correlation ID and event ID to every message.
10. Test producing and consuming a sample `job.scrape.commands` event before adding the scraper.

For a small daily workload, start with one or a small number of partitions per topic. Increase partitions only when parallelism or throughput requires it.

### ### Stage 3 â€” Event-driven scraping

1. The timer producer loads enabled companies from configuration.
2. It publishes one `ScrapeCompanyCommand` per company to `job.scrape.commands`.
3. The scraper consumer receives a command and selects the matching `CompanyScraper` implementation.
4. The company scraper fetches search pages and job-detail pages using Java HTTP Client.
5. Jackson parses the JSON envelope and Jsoup parses the embedded HTML.
6. Raw search responses and detail HTML are written to Blob Storage.
7. The consumer publishes `job.scrape.completed` with the run ID, company ID, counts, errors, and Blob paths.
8. Failed work is retried and eventually sent to the dead-letter topic.

For McKesson, use `data-job-id` as the stable external ID. Important response metadata includes `data-total-job-results`, `data-total-pages`, `data-current

### ### Stage 4 â€” Blob Storage and Cosmos DB

Blob layout:

```
```text
raw/{companyId}/{yyyy}/{MM}/{dd}/{runId}/search-page-1.json
raw/{companyId}/{yyyy}/{MM}/{dd}/{runId}/job-{externalJobId}.html
normalized/{companyId}/{yyyy}/{MM}/{dd}/{runId}/run-summary.json
```
```

Use one Cosmos database with three containers:

#### #### `job-postings`

One current normalized document per posting. The partition key: `/companyId`.

```
```json
{
  "id": "mckesson-99215825472",
  "companyId": "mckesson",
  "externalJobId": "99215825472",
  "titleRaw": "Lead Data Architect (Healthcare)",
  "titleNormalized": "Data Architect",
  "jobLevel": "Lead",
  "location": "Irving, TX",
  "datePosted": "2026-08-14",
  "firstSeenAt": "2026-08-15T10:00:00Z",
  "lastSeenAt": "2026-08-15T10:00:00Z",
  "isActive": true,
  "contentHash": "sha256:...",
  "schemaVersion": 1
}
```
```

#### #### `job-observations`

One document per posting observed per daily run. Store `companyId`, `jobId`, `observedAt`, `datePosted`, normalized title, job level, content hash, and scr

#### #### `monthly-metrics`

Precomputed documents keyed by company, month, normalized title, and job level. The partition key: `/month`.

```
```json
{
  "id": "mckesson-2026-08-data-architect-lead",
  "companyId": "mckesson",
  "month": "2026-08",
  "normalizedTitle": "Data Architect",
  "jobLevel": "Lead",
  "postingCount": 4,
  "uniquePostingCount": 4
}
```
```

The persistence consumer should be idempotent. Use the event ID, company ID, and external job ID to prevent duplicate writes when Kafka redelivers an event

### ### Stage 5 â€” Monthly metrics

1. Consume `job.postings.normalized` in a dedicated metrics consumer group.

2. Decide whether the metric counts by `datePosted` or `firstSeenAt`.
3. Normalize titles consistently before aggregation.
4. Derive a job level using explicit rules and retain the raw title.
5. Upsert a monthly metric document using company, month, title, and level.
6. Publish a `job.metrics.monthly` event after successful updates.
7. Rebuild a month from `job-observations` when normalization rules change.

### ### Stage 6 – Daily email with Twilio SendGrid

1. Consume completed scrape and metrics events.
2. Build a daily summary containing companies processed, new postings, changed postings, inactive postings, failures, top titles, and top job levels.
3. Publish a `notifications.daily` event with a unique run ID.
4. Consume that event in a dedicated email consumer group.
5. Send the message with the official SendGrid Java library.
6. Store the API key as an Azure Function or Spring application setting.
7. Retry temporary failures and prevent duplicate emails for the same run ID.

The SendGrid free trial is limited and should be treated as a development allowance, not a permanent free tier.

### ### Stage 7 – Infrastructure as Code for Azure and Confluent

Use a hybrid infrastructure automation approach:

- **Bicep** manages Azure-native resources.
- **Terraform** manages Confluent Kafka resources.
- **GitHub Actions** orchestrates validation, planning, approval, deployment, and smoke tests.

**Bicep (Azure resources):**

1. Create `infra/azure/main.bicep` with modules for Function App, Storage, Cosmos DB, Key Vault, and monitoring.
2. Parameterize resource names, locations, SKUs, and environment-specific settings.
3. Use managed identities for inter-resource and Azure-to-Confluent authentication.
4. Outputs should expose connection strings, endpoint URIs, and identity details for app settings wiring.

**Terraform (Confluent Kafka resources):**

1. Create `infra/confluent/main.tf` to manage the Confluent broker, topics, service accounts, API keys, and ACLs.
2. Parameterize cluster details, topic partition counts, replication factors, and retention policies.
3. Use Terraform Cloud or Azure Storage backend for remote state.
4. Create separate service accounts for scraper, normalizer, metrics, and email consumers.
5. Apply fine-grained ACLs so each consumer group has read access only to its required topics.

**GitHub Actions CI/CD pipeline:**

1. Validate Bicep syntax and Terraform format on every pull request.
2. Run `terraform plan` and capture Confluent resource diffs.
3. Run `az bicep build` or `what-if` to preview Azure changes.
4. Require manual approval before applying infrastructure changes to production.
5. Apply Bicep to deploy Azure resources.
6. Apply Terraform to create/update Confluent resources.
7. Deploy the Java Function App artifact and update app settings.
8. Run a smoke test that publishes a test scrape command and verifies the expected event path.

**Repository layout:**

```

...
infra/
 azure/
 main.bicep
 parameters.json
 modules/
 function-app.bicep
 storage.bicep
 cosmos.bicep
 keyvault.bicep
 monitoring.bicep
 confluent/
 main.tf
 variables.tf
 outputs.tf
 terraform.tfvars.example
 modules/
 cluster/
 topics/
 service-accounts/
 acls/
.github/
 workflows/
 deploy.yml
...

```

**Key practices:**

- Store Confluent API credentials in GitHub Secrets, never in Terraform files.
- Use Azure Key Vault to store Kafka credentials at runtime; inject them into Function App settings during deployment.
- Keep infrastructure state remote (Terraform Cloud or Azure Storage backend).
- Version both Bicep and Terraform templates in Git.
- Run infrastructure and app deployment as separate steps so app releases do not depend on infrastructure changes.

### ### Stage 8 – GitHub Actions CI/CD for application and infrastructure deployment

Pipeline:

```

...
validate
 -> unit-tests
 -> package
 -> plan-azure
 -> plan-confluent
 -> approval-gate
 -> apply-azure
 -> apply-confluent
 -> deploy-app
 -> smoke-test-kafka

```

...

1. Build and test the Maven project on pull requests.
2. Validate topic names, consumer groups, and configuration templates.
3. Build the Azure Functions and Spring Boot artifacts.
4. Run `terraform plan` for Confluent resources.
5. Run Bicep `build` or Azure `what-if` for Azure resources.
6. Require manual approval before deployment to production.
7. Deploy Bicep for the Function App, Storage, Cosmos DB, Key Vault, and monitoring.
8. Deploy Terraform for Kafka topics, service accounts, and ACLs.
9. Deploy application code after infrastructure succeeds; update app settings and wired secrets.
10. Store Kafka, Cosmos, and SendGrid secrets in GitHub Secrets and Azure Key Vault, never in code.
11. Restrict production deployment to the main branch.
12. Run a smoke test that publishes a test command and verifies the expected event path.
13. Add a manual replay workflow for a selected company and date range.

## Updated the implementation order

1. Create and build the Maven Azure Functions project.
2. Add Spring Boot application structure.
3. Configure the existing Confluent broker and create Kafka topics.
4. Publish and consume a test scrape command.
5. Implement the McKesson scraper consumer.
6. Add Blob Storage raw persistence.
7. Add normalization and Cosmos DB persistence.
8. Add multiple company configurations.
9. Add monthly metrics.
10. Add daily SendGrid summaries.
11. Create Bicep templates for Azure infrastructure (Stage 7).
12. Create Terraform modules for Confluent resources (Stage 7).
13. Build GitHub Actions CI/CD pipeline (Stage 8).

## Single Function App deployment

Use one Flex Consumption Function App containing five independently triggered functions:

```
```text
job-scraper-function
|- DailyScrapeCommandProducer      Timer trigger
|- ScraperConsumer                 Kafka trigger
|- NormalizerConsumer              Kafka trigger
|- MetricsConsumer                 Kafka trigger
|- SendGridEmailConsumer           Kafka trigger
```
```

The Function App is the deployment boundary. Kafka topics, consumer groups, partitions, retries, and dead-letter topics provide the event-driven behavior.

This is a serverless, event-driven modular platform with microservice-style bounded components. It should not be described as independently deployable micr

## Architecture guarantees

### Job deduplication

Kafka processing must be treated as at-least-once delivery. Consumers must be idempotent.

Use deterministic identifiers:

```
```text
Current posting ID:
{companyId}:{externalJobId}

Observation ID:
{companyId}:{scrapeDate}:{externalJobId}

Scrape command ID:
{runId}:{companyId}
```
```

The job-postings Cosmos document uses the company ID and external job ID as its stable identity:

```
```json
{
  "id": "mckesson:99215825472",
  "companyId": "mckesson",
  "externalJobId": "99215825472"
}
```

The persistence consumer uses upsert operations. Replayed events update the existing document rather than creating duplicates. Daily observation documents

Consumer processing order:

```
```text
Receive Kafka event
-> validate event
-> perform deterministic persistence
-> publish downstream event
-> commit Kafka offset
```
```

Offsets must not be committed before successful persistence.

Parallel scraping and failure isolation

Publish one ScrapeCompanyCommand per company. Never publish one message containing the complete company list.

```
```text
job.scrape.commands
```



```

ScrapeCompany(USAA)
ScrapeCompany(GM Financial)
ScrapeCompany(BNSF)
ScrapeCompany(Oracle)
ScrapeCompany(Microsoft)
ScrapeCompany(Amazon)
ScrapeCompany(Google)
ScrapeCompany(Lockheed Martin)
ScrapeCompany(Forescout)
ScrapeCompany(Indeed)
...

```

Use companyId as the Kafka message key. Kafka routes records with the same key to the same partition, preserving ordering for that key. Multiple partitions

Use one-message processing for the scraper command topic initially. This ensures a failure for Oracle does not cause a batch containing successful USAA, GM

Expected behavior:

```

...`text
USAA -> SUCCESS -> commit offset
GM -> SUCCESS -> commit offset
BNSF -> SUCCESS -> commit offset
Oracle -> FAILURE -> retry Oracle only
Microsoft -> SUCCESS -> commit offset
...

```

Each scraper should have its own timeout, retry limit, backoff policy, rate limit, circuit breaker, failure status, and dead-letter handling.

After the retry limit is exceeded:

```

`text
ScrapeCompany(Oracle)
 -> retry with backoff
 -> job.scraper.commands.DLT
 -> ScrapeCompanyFailed event
`

```

A failed company must not prevent other company commands from completing.

### Pluggable scraper implementations

Use a common scraper contract and registry:

```

...`java
public interface CompanyScraper {
 CompanyId companyId();
 ScrapeResult scrape(ScrapeContext context);
}
...

```

Each company implements the contract independently:

```

...`text
CompanyScraper
|- UsaaScraper
|- GmFinancialScraper
|- BnsfScraper
|- OracleScraper
|- MicrosoftScraper
|- AmazonScraper
|- GoogleScraper
|- LockheedMartinScraper
|- ForescoutScraper
`- IndeedScraper
...

```

The Kafka consumer selects a scraper through CompanyScraperRegistry. It must not contain company-specific if or switch statements.

Adding a company should require:

```

...`text
Add NewCompanyScraper
Add parser tests
Add saved response fixtures
Add company configuration
No changes to Kafka consumer
No changes to persistence
No changes to metrics
...

```

## Code structure

Use a modular hexagonal structure so the single Function App can later be split into multiple deployable services without rewriting business logic:

```

`text
src/main/java/com/jobscraper/
|- function/
| |- DailyScrapeCommandProducerFunction.java
| |- ScraperConsumerFunction.java
| |- NormalizerConsumerFunction.java
| |- MetricsConsumerFunction.java
| `-- SendGridEmailConsumerFunction.java
|- application/
| |- command/
| |- normalization/
| |- metrics/
| `-- notification/

```

```

|- domain/
| |- company/
| |- job/
| |- events/
| `-- metrics/
|- scraper/
| |- api/
| |- registry/
| `-- companies/
| |- usaa/
| |- gmfinancial/
| |- bnsf/
| |- oracle/
| |- microsoft/
| |- amazon/
| |- google/
| |- lockheedmartin/
| |- forescout/
| `-- indeed/
|- infrastructure/
| |- kafka/
| |- blob/
| |- cosmos/
| |- email/
| `-- observability/
`-- configuration/

```

Tests should mirror the company adapters and application handlers:

```

`text
src/test/java/com/jobscraper/
|- scraper/companies/
|- application/
|- infrastructure/
`-- contract/

```

Each scraper should have saved HTML or JSON fixtures so parser tests are deterministic and resilient to website changes.

### ## Portfolio architecture statement

This project can accurately be described as:

> A serverless, event-driven job-ingestion platform where each company is a pluggable scraper adapter. Daily commands are distributed through Kafka, compar

The single Function App optimizes deployment simplicity while retaining:

- Serverless execution
- Event-driven processing
- Asynchronous Kafka communication
- Partition-based parallelism
- Fault isolation
- Retry and dead-letter handling
- Idempotent persistence
- Pluggable scraper adapters
- Historical analytics
- Observability

### ## Official references

- [Azure Java Functions](<https://learn.microsoft.com/en-us/azure/azure-functions/functions-reference-java>)
- [Azure Timer Trigger](<https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-timer>)
- [Azure Storage libraries for Java](<https://learn.microsoft.com/en-us/java/api/overview/azure/storage?view=azure-java-stable>)
- [Azure Cosmos DB Java SDK v4](<https://learn.microsoft.com/en-us/azure/cosmos-db/sdk-java-v4>)
- [Azure Functions GitHub Actions](<https://learn.microsoft.com/en-us/azure/azure-functions/functions-how-to-github-actions>)
- [Twilio SendGrid Java quickstart](<https://www.twilio.com/docs/sendgrid/for-developers/sending-email/email-quickstart-for-java>)

Design reference date: August 16, 2026.